Smart Text Analyzer

Introduction

The Smart Text Analyzer is an interactive Python based application that aims to assist users in efficiently analyzing and understanding their text, the purpose is to provide insights such as word and character statistics, keyword extraction, sentiment analysis , and intelligent text suggestions

Users can insert the text directly or they could load it from a file. The program then will clean and standardize the text for accurate analysis and presents a simple and user friendly menu to choose operations

The application includes intelligent features such as smart autocompletion , next word prediction , sentiment analysis with negation handling, and keyword extraction. These features use efficient data structures such as dictionaries, sets, and Tries to
. provide fast and accurate results

Overall, the Smart Text Analyzer helps users gain meaningful insights from text while ensuring ease of use and responsiveness.

Objective

This project aims to provide intelligent text analysis using Python's text processing techniques, including word statistics, word prediction, and sentiment analysis.

Features Available to the User:

1. Word Statistics (Total & Unique)
2. Search for a Word/Phrase
3. Word Replacement
4. Character Statistics
5. Keyword Extraction
6. Sentiment Analysis
7. Smart Autocomplete
8. Next Word Prediction
9. Undo Last Replacement

General architectural

Main Loop:

The program runs in an infinite loop presenting a menu of options until the user chooses to exit.

Libraries/Modules/Functions:

- Libraries :
    1. string : To obtain a list of symbols (punctuation)
    2. collections : For word statistics (Counter To calculate word frequency with an O(1) lookup and defaultdict To automatically generate lists and dictionaries without needing to check for the existence of a key and To facilitate the construction of Trie and N-gram models)
- **Functions :**
    1. preprocess_text() : Lowercase, remove punctuation, split into words/sentences.
    2. build_bigram_counts() / build_trigram_counts() : Build N-gram models.
    3. predict_next_word() : Suggest next words using N-grams.
    4. Trie : Autocomplete feature.

Menu options 1–10 handle each feature individually.

Core Data Structures Table

Data Structure	Type / Implementation	Purpose / Reason	Time Complexity
words_list, sentences_list	List	Store all words and sentences. Allows easy iteration, indexing, and processing by all functions.	O(n) for iteration/search
word_counts	Dictionary / Counter	Track word frequencies. Provides O(1) average lookup and insertion.	O(1) average for insert / lookup
unique_words	Set	Store unique words. Fast membership check (O(1)) for removing duplicates.	O(1) average lookup
bigram_counts, trigram_counts	defaultdict of defaultdict(int)	Store N-gram occurrences. Nested dictionaries allow automatic frequency counting without key errors.	O(n) to build, O(1) lookup per next word
stop_words, positive_words, negative_words, negation_words	Set	Efficient lookups for filtering words and sentiment analysis.	O(n) to build, O(1) lookup per next word

trie	Trie Tree (nodes as defaultdict)	Prefix-based autocomplete. Fast suggestions based on prefixes.	O(k) insert/search per word ,k = length of word; prefix search fast, independent of total words
undo_stack	List (used as Stack)	Store previous versions of text after edits. LIFO behavior allows undoing last modifications efficiently.	O(1) push/pop
wordQueue	Queue	Process words sequentially (FIFO) for	O(n) to build &
Data Structure	Type / Implementation	Purpose / Reason	Time Complexity
	(collections.deque)	Keyword Extraction after that filter stop words and count frequency	process

Implementation Details

Text input and processing

Initially, we created a function for the user to input text, providing two input methods: the first by uploading a file, and the second by the user directly entering the text. Finally, the user enters the phrase $$END TEXT$$ to complete the input process.

After the user enters the text, it undergoes a processing operation where all text is converted to lowercase, punctuation is removed, and finally, the text is converted from a string to a list for easier manipulation.

The goal of the processing operation: to standardize the data and prepare it for analysis.

Core Features

1. Word Statistics:

   In this Feature we Calculate and display the total number of words.,Calculate and display the number of unique words (case-insensitive).,Display the frequency of each unique word, perhaps top N most frequent. , For example :

   ```
   Total Number of Words: 301

   Number of Unique Words: 157

   Top 5 most frequent words: [('the', 66), ('in', 13), ('and', 9), ('they', 8), ('a', 7)]
   ```

   Big O : Counting words: O(n) ,Top 5 frequent: O(u log u) if sorting, O(u) with Counter.most_common()

2 Character Statistics:

In this Feature we Count total characters (excluding spaces)., Count occurrences of each character. using Counter , For example ;

```
Total characters (excluding spaces): 1283
```

```
Character occurrences: {'t': 133, 'h': 118, 'e': 184, 'q':
2, 'u': 29, 'i': 67, 'c': 31, 'k': 14, 'b': 17, 'r': 83,
'o': 81, 'w': 33, 'n': 82, 'f': 26, 'x': 1, 'j': 1, 'm': 16,
'p': 21, 's': 76, 'v': 8, 'l': 56, 'a': 74, 'z': 5, 'y': 25,
'd': 67, 'g': 33}
```

Big O : O(n)

3 Search Functionality:

In this Feature we Allow the user to search for a specific word or phrase. Report all occurrences, including their positions (e.g., sentence number,word index within sentence). By using Iterates sentences using enumerate() to get the tracks positions , For example ;

```
Enter the word to search for: the
```

```
Found in sentence 1, word position 1
```

```
Found in sentence 1, word position 7
```

```
Found in sentence 1, word position 16…
```

Big O : Worst case: O(n * m) where n = number of sentences ,m = average words per sentence.

4 **Word Replacement:**

Enable the user to replace all occurrences of a specific word with

another. Confirm the change.

Save previous version in undo_stack : can undo last modification.and Rebuild N-grams & counters after replacement , For example :

```
Word to replace: the
```

```
New word: a
```

```
Successfully replaced all occurrences of 'the' with 'a'.
```

Big O : O(n) to iterate and replace words, Rebuild bigram/trigram: O(n)

Intelligent Features

1. Keyword Extraction

This feature removes the stop words , then processes the remaining words sequentially using a queue (FIFO) to count the frequencies , and finally returns the top 5 keywords , For example :

```
--- Top 5 Keywords ---
fell -> 4
grew -> 4
day -> 3
trees -> 3
ground -> 3
```

Big O :Filtering and counting : O(n), Sorting by frequency: O(u log u)

2. Sentiment Analysis

Determine the overall sentiment (positive, negative, or neutral) of a user-provided sentence or a sentence extracted from the loaded text, For example :

```
Enter a sentence to analyze sentiment: i am so happy
Sentiment: Positive
Enter a sentence to analyze sentiment: this project not bad
Sentiment: Positive
```

Big O : O(m) per sentence, m = words in sentence, Lookup in positive/negative/negation sets: O(1) per word

3. **Smart Autocomplete**

Suggest words based on prefix, Top 5 suggestions sorted by frequency Trie stores words alphabetically to facilitate quick word completion. Basic operations: Add a word, search, suggest words by prefix , Each operation is O(n) where n is the word length, For example

```
Enter a prefix: ap
Suggestions: ['apple']
Enter a prefix: d
Suggestions: ['day', 'down', 'dog', 'dusty', 'days']
```

Big O :Trie search: O(k + t log t) ,k = length of prefix, t = number of suggestions (sort by frequency)

4 Next Word Prediction

Uses Trigram model first, fallback to Bigram, Sorted by frequency, return top N predictions

N-grams :Relies on word sequence to predict the next word (Bigram/Trigram). Based on the Markov Assumption and the Multiple Enumeration of Possibilities (MLE). Used to provide intelligent suggestions for the next word., For example :

```
Enter a word or phrase: sang

Next word suggestions: ['their', 'songs']
```

Big O : Lookup in bigram/trigram dicts: O(1) per next word suggestion , Sorting top_n: O(s log s), s = number of candidate next words

5 Undo Last Replacement

Reverting the text to its state before the last word replacement allows the user to undo any unwanted changes, For example :

```
Select an option (1-9): 9

Last modification undone successfully.
```

Big O : Push/Pop from undo_stack: O(1) , Rebuilding models: O(n)

Challenges & Solutions

Challenge 1 : Maintaining Autocomplete after Word Replacement or Undo Initially, when we completed the word replacement and undo features , we came across a technical issue : after replacing a word and then using undo , the smart autocomplete feature would still suggest words based on the old version of the text , this happened because the Trie which stores the words for autocomplete, was not rebuilt after the undo operations, so the solution was updating the code to rebuild the Trie after every word replacement and undo, this ensures that autocomplete suggestions always reflect the current state of the text.

Challenge 2: Handling Negations in Sentiment Analysis and Efficient Keyword Extraction Another challenge faced was text preprocessing for intelligent features.

For example, sentiment analysis, words like "not good" or "never happy" , the solution was to use set of negation words in the code, so one of the set values comes before a positive word it would be count as negative and vice versa.

similarly, for keyword extraction, efficient removal of stop words had to be achieved while processing words sequentially for large text, solution : a queue was used to process words sequentially, skipping stop words, and then using a counter for efficient keyword extraction.

Future Work

If we had more time, we could enhance the Smart Text Analyzer in many ways like :

1. Advanced Sentiment Analysis : Implement a more complex sentiment analysis using machine learning to detect other situations such as sarcasm, bullying, context, and emotions beyond simple lexicon based scoring .

2. Phrase and multi word Keyword Extraction : improve the keyword extraction feature to detect multi words expressions or significant phrases, not only a single word, for more meaningful insights .

3. Text Summarization feature : Add a feature to summarize long texts immediately, to give the users a brief idea of the content.

4. Graphical Interface : Develop a graphical interface to make the program easier in interaction, making it accessible to users who prefer not to work with the command line interface.